A Unified Projection Framework for Tensor-Structured Dictionary Learning: Benchmarking CP, Tucker, Tensor-Train, Tensor-Ring and Block-Term Decompositions, with an Efficiency Ranking System

Technical Report

July 21, 2026

Abstract

Dictionary learning represents signals as sparse combinations of learned atoms, but for multiway data (images, video, hyperspectral cubes) the standard approach vectorizes each patch, discarding spatial/spectral structure and inflating the dictionary to $\prod_i n_i \times K$ free parameters. Tensor decompositions repair this by constraining the *atoms* to a low-parameter format. We place five tensor-structured dictionaries—Canonical Polyadic (CP), Tucker, Tensor-Train (TT), Tensor-Ring (TR) and Block-Term Decomposition (BTD)—together with an unstructured baseline inside a *single* learning harness: identical Orthogonal Matching Pursuit (OMP) sparse coding and identical reconstruction operate on a shared *effective dictionary*, so that the only thing that varies between methods is how each atom is parameterized and how many parameters that costs. Four of the six methods differ only by a projection ("snap") step inside the K-SVD atom update; Tucker is realized as a Kronecker dictionary with a separable alternating-least-squares update. We benchmark on synthetic mixed-structure patches, real RGB image patches, and a hyperspectral cube, and we build a three-layer efficiency ranking system (Pareto dominance, an interpretable accuracy-per-parameter scalar, and profile-weighted composite tiers). Our central finding is that the "best" decomposition is strongly regime-dependent: the unstructured baseline wins on rich clean synthetic data; structured dictionaries win on noisy real patches, where structure acts as a regularizer; and at *matched parameter budget* on the hyperspectral cube the ranking neither favours Tensor-Train despite a large third mode, nor flips with noise. We also show that a fairness correction to the Tucker optimizer moves it from last place to a Pareto-dominant winner ($30.7 \rightarrow 32.2$ dB at identical size), a concrete demonstration that implementation confounds can dominate structural differences. We report all numbers, release the harness, and are explicit about the limitations of comparing projection variants rather than canonical published algorithms.

1 Introduction

Classical dictionary learning (DL) models a signal $y \in \mathbb{R}^n$ as $y \approx Dx$, where $D \in \mathbb{R}^{n \times K}$ is an overcomplete dictionary whose columns are unit-norm *atoms* and x is sparse. K-SVD [1] learns D by alternating sparse coding with a rank-one singular-value update of each atom. For multiway signals—an $n_1 \times n_2 \times n_3$ image or hyperspectral patch—the standard recipe flattens each patch to a vector of length $n = \prod_i n_i$. This vectorization has two costs. First, it discards the multiway structure (spatial neighbourhoods, spectral smoothness) that makes the data compressible. Second, the dictionary carries $n \cdot K$ free numbers, which grows multiplicatively with tensor order and invites over-fitting.

Tensor decompositions [3] address both problems by constraining atoms (or the dictionary as a whole) to a low-parameter tensor format. The literature contains many such constructions—separable/Kronecker dictionaries [8, 9], CP-structured atoms, and tensor-network dictionaries—but they are usually studied in isolation, each with its own optimizer, which makes cross-method comparison unreliable: an observed gap may reflect the optimizer or the hand-picked settings rather than the decomposition. Our goal is a *controlled* comparison.

Contributions. (1) A unified harness in which CP, Tucker, TT, TR and BTD dictionaries, plus an unstructured baseline, share the same OMP sparse coding, the same reconstruction, and the same evaluation, so only the atom structure differs (Section 2). (2) A per-atom *projection* view that realizes CP/TT/TR/BTD as one line changed inside K-SVD, and a matched in-harness Tucker via separable alternating least squares (Section 3). (3) A three-layer efficiency ranking system—objective Pareto dominance, an interpretable accuracy-per-parameter scalar, and profile-weighted tiers (Section 4). (4) Experiments on synthetic, real RGB and hyperspectral data, including a matched-parameter-budget rate–distortion study across noise levels, with an explicit account of when and why the ranking changes (Sections 5–7).

2 A Unified Framework for Tensor Dictionary Learning

2.1 The shared objective and the effective dictionary

Given training patches $\{\mathcal{Y}_m\}_{m=1}^M$, every method solves the same sparse-coding objective on the vectorized patches $y_m = \mathrm{vec}(\mathcal{Y}_m) \in \mathbb{R}^n$,

$$\min_{D,\{x_m\}} \sum_{m=1}^{M} \|y_m - Dx_m\|_2^2 \quad \text{s.t.} \quad \|x_m\|_0 \le s, \quad \|d_k\|_2 = 1, \tag{1}$$

where $D = [d_1, \ldots, d_K]$ collects the atoms. The methods differ *only* in how the columns of the *effective dictionary* D are parameterized. Because Eq. (1) and its solver are shared, the accuracy comparison is apples-to-apples: sparse coding uses Orthogonal Matching Pursuit (OMP) [2] against D, and reconstruction is the linear map $x_m \mapsto Dx_m$, for all methods identically.

2.2 Where a decomposition can enter

A tensor decomposition can constrain the dictionary in two structurally different places, and this distinction organizes everything that follows.

- **Structure on each atom** (CP, TT, TR, BTD). Each atom is *itself* a low-parameter tensor. Since atoms are independent, the constraint is enforced atom-by-atom, by projecting the optimal unconstrained atom onto the decomposition's manifold inside the K-SVD update.
- **Structure on the whole dictionary** (Tucker). Every atom is built from a shared set of per-mode vectors, so the constraint couples all atoms; there is no single atom to project, and the update re-fits the shared mode factors instead.

2.3 Two algebraic bridges: Khatri–Rao and Kronecker

The link between a decomposition and the matrix form $Y \approx DX$ of DL runs through two products.

CP ⇒ Khatri–Rao. A rank-one atom $a \circ b \circ c$ vectorizes to the Kronecker product of its factors. If atom k is built from column k of factor matrices A, B, C, then column k of the dictionary is $a_k \otimes b_k \otimes c_k$, so the entire dictionary is the column-wise Khatri–Rao product

$$D \;=\; A \odot B \odot C. \tag{2}$$

CP-DL therefore learns the small factors A, B, C; the atoms are the matched triples; and X is an ordinary sparse matrix selecting a few of the K atoms. Storage is $(n_1{+}n_2{+}n_3)K$ rather than $n_1 n_2 n_3\, K$.

Tucker ⇒ Kronecker. A Tucker signal $\mathcal{Y} = \mathcal{G} \times_1 U_1 \times_2 U_2 \times_3 U_3$ vectorizes to

$$\mathrm{vec}(\mathcal{Y}) \;=\; (U_3 \otimes U_2 \otimes U_1)\ \mathrm{vec}(\mathcal{G}), \tag{3}$$

which is exactly $y = Dx$ with $D = U_3 \otimes U_2 \otimes U_1$ (the full Kronecker product) and $x = \mathrm{vec}(\mathcal{G})$ the (sparse) core. The Khatri–Rao is the column-matched Kronecker: CP keeps only matched triples (K atoms, a free sparse code), while Tucker keeps *all* $k_1 k_2 k_3$ combinations, with the core playing the role of the sparse code.

3 Methods

3.1 Dictionary learning Background

3.2 The projection view of the K-SVD atom update

Given the training signals, the singular value decomposition points to the single direction of strongest correlation in the error an atom is responsible for, and takes that direction as the updated atom. K-SVD builds the dictionary by alternating two steps: a sparse-coding step, in which orthogonal matching pursuit (§3.3) decides which atoms each signal uses and with what coefficients, and a dictionary-update step, which refines the atoms one at a time with the codes held fixed. This subsection concerns the update step; we first state it as a rank-one approximation (§3.2.1) and then make precise the matrix being approximated (§3.2.2).

3.2.1 Rank-one approximation

Updating a single atom together with the coefficients that use it is a rank-one approximation problem. Let E be the error matrix collecting what atom k must explain across the signals that use it; its construction is given in §3.2.2. The optimal rank-one update is the leading singular pair of E: the new (unconstrained) atom is u_1 and the new codes are $\sigma_1 v_1$, where $E = \sum_i \sigma_i u_i v_i^\top$ is the SVD.

For a matrix atom this leading singular pair is precisely the best rank-one (CP) term — which is why the SVD *is* the CP decomposition in two dimensions. For a genuine order-N atom, no single SVD yields the best rank-one tensor, and CP alternating least squares supplies the higher-order analogue.

Our four per-atom methods keep this loop and insert one projection: the optimal atom u_1 is reshaped to a tensor and replaced by its nearest point on a chosen manifold,

$$d_k \;\leftarrow\; \Pi_{\mathcal{M}}\big(\mathrm{reshape}(u_1)\big), \qquad \mathcal{M} \in \{\text{free},\ \text{CP},\ \text{TT},\ \text{TR},\ \text{BTD}\}. \tag{4}$$

The identity projection ($\mathcal{M} = $ free) recovers ordinary K-SVD, the vectorized baseline. The others use rank-R CP via alternating least squares; TT-SVD at ranks $[1, r, \ldots, r, 1]$; tensor-ring at uniform

rank r; and BTD via greedy fitting of b multilinear-rank-(L, L, L) Tucker blocks. BTD interpolates the family: with $L = 1$ it reduces to CP rank-b, and with a single full-rank block it reduces to Tucker.

3.2.2 The restricted error matrix

The matrix E approximated in §3.2.1 is not the full residual but a *restricted* one. To update atom k, let Ω be the set of signals that use it, and form the error those signals still assign to atom k,

$$E = Y_\Omega - DX_\Omega + d_k\, x_{k,\Omega}^\top. \tag{5}$$

The term $Y_\Omega - DX_\Omega$ is the residual of the current dictionary on the selected signals, and adding back $d_k\, x_{k,\Omega}^\top$ removes atom k's own contribution, leaving exactly the error that a new d_k must account for. Restricting to Ω is what preserves the sparsity pattern: the update is computed only over signals that already use the atom, so the rank-one step never introduces the atom into signals whose codes were zero. This is the "restricted" in restricted error matrix, and it is the property that lets the atom update and the sparse code stay consistent from one iteration to the next.

3.3 Orthogonal matching pursuit

Orthogonal matching pursuit (OMP) is the sparse-coding step of the loop: although presented here after the update, it is what runs first in each K-SVD iteration to produce the codes X that (5) then holds fixed. For a single signal y and dictionary $D = [d_1, \ldots, d_M]$, OMP selects atoms one at a time and, after every selection, re-solves for *all* chosen coefficients by least squares, so the residual is always orthogonal to the span of the atoms picked so far. That orthogonality is what distinguishes OMP from plain matching pursuit and prevents an atom from being selected twice.

3.3.1 Greedy atom selection

Starting from the full signal as the residual, $r_0 = y$, and an empty support $\Lambda_0 = \varnothing$, each iteration measures how well every atom aligns with the part of the signal not yet explained. It correlates each atom with the current residual, $c_j = |\langle d_j, r_{k-1}\rangle|$, and greedily adds the single best-aligned atom to the support,

$$\lambda_k = \arg\max_j c_j, \qquad \Lambda_k = \Lambda_{k-1} \cup \{\lambda_k\}. \tag{6}$$

Because the residual is kept orthogonal to the already-selected atoms (§3.3.2), those atoms have zero correlation and cannot be chosen again, so each step commits to a genuinely new atom.

3.3.2 Residual update

Rather than keep the coefficients from earlier iterations, OMP recomputes the entire coefficient vector on the active set D_{Λ_k} by least squares at every step,

$$\hat{x} = \arg\min_x \|y - D_{\Lambda_k}\, x\|_2, \qquad r_k = y - D_{\Lambda_k}\, \hat{x}. \tag{7}$$

This orthogonal projection is the defining line of the algorithm: the residual r_k is orthogonal to every selected atom, so the next correlation measures only what the current approximation has not yet explained. The loop stops once the residual falls below the tolerance, $\|r_k\|_2 < \varepsilon$, or the sparsity cap $|\Lambda_k| = s$ is reached. The codes $\hat{x}$ gathered over all signals form the matrix X that the dictionary-update step of §3.2 then holds fixed while it refines the atoms; the two steps alternate until convergence.

Algorithm 1 Orthogonal Matching Pursuit (sparse coding step)

Require: dictionary $D = [d_1, \ldots, d_M]$, signal y, tolerance ε, sparsity cap s
Ensure: sparse code $\hat{x}$ with support Λ

Initialize: residual $r_0 \leftarrow y$, support $\Lambda_0 \leftarrow \varnothing$, $k \leftarrow 0$
repeat
  $k \leftarrow k + 1$
  Correlate: $c_j = |\langle d_j, r_{k-1} \rangle|$ for all atoms j ▷ §3.3.1
  Select: $\lambda_k = \arg\max_j c_j$
  Augment support: $\Lambda_k = \Lambda_{k-1} \cup \{\lambda_k\}$
  Project: $\hat{x} = \arg\min_x \|y - D_{\Lambda_k} x\|_2$ ▷ §3.3.2
  Update residual: $r_k = y - D_{\Lambda_k} \hat{x}$
until $\|r_k\|_2 < \varepsilon$ **or** $|\Lambda_k| = s$
return $\hat{x}$, Λ_k

3.4 Tucker in the same harness

Because Tucker's constraint is shared across atoms, it cannot be projected atom-by-atom. We keep the same outer loop—OMP sparse coding against $D = U_3 \otimes U_2 \otimes U_1$, giving each sample a sparse core—then update the mode factors by separable alternating least squares. Holding the cores fixed, each U_n minimizes $\sum_m \|\mathcal{Y}_m - \mathcal{C}_m \times_1 U_1 \times_2 U_2 \times_3 U_3\|^2$; for mode n this is the linear solve $U_n = X_n G_n^\top (G_n G_n^\top + \lambda I)^{-1}$, where $G_m = \mathcal{C}_m \times_{p \neq n} U_p$. This "re-fit the shared factors" step is the Tucker analogue of the per-atom snap and is the canonical separable/Kronecker dictionary construction [8].

3.5 Parameter accounting

Table 1 lists the free parameters stored per method for an order-N atom of shape $(n_1, \ldots, n_N)$ with $S = \sum_i n_i$. The dictionary parameter count is the storage budget we compare on; the sparse code x_m (or core) is per-sample and excluded from all methods equally.

Method	Dictionary form	Parameters
Vectorized	free atoms	$K \prod_i n_i$
CP (rank R)	$D = A \odot B \odot C$ (Khatri–Rao)	KRS
Tucker	$D = U_N \otimes \cdots \otimes U_1$ (Kronecker)	$\sum_i n_i k_i$
TT (rank r, N=3)	chain of cores	$K\left[r(n_1+n_3) + r^2 n_2\right]$
TR (rank r)	ring of cores	$Kr^2 S$
BTD (L, b blocks)	sum of Tucker blocks	$Kb\,(L^N + LS)$

Table 1: Dictionary parameter cost per method. K is the number of atoms; for Tucker the atom count is $\prod_i k_i$ and the stored parameters are only the mode factors.

3.6 Sparse coding from a CP viewpoint

OMP builds each sparse code greedily: start with residual $r = y$; score every atom by its correlation $\langle d_k, r \rangle$; select the largest in magnitude; re-fit the weights of all selected atoms by least squares; update the residual; repeat until s atoms are chosen. For CP atoms the score factorizes, $\langle a \circ b \circ c, \mathcal{R} \rangle = \mathcal{R} \times_1 a \times_2 b \times_3 c$: scoring is a separable multilinear contraction of the residual with the atom's factor

vectors, so the full atom need never be formed. The collection of selections and weights, one sparse column per signal, is the sparse matrix X.

4 Efficiency Ranking System

"Efficiency" of a decomposition for DL is multi-objective—accuracy, parameter thrift, training speed, and robustness—so we rank in three layers from objective to opinionated.

Layer 1: Pareto dominance (weight-free). On the two core axes, test accuracy and parameter-efficiency, a method is *dominated* if another is at least as good on both and strictly better on one; no non-negative weighting can rescue a dominated method. The non-dominated methods form the rate–distortion frontier.

Layer 2: an interpretable scalar. $E = \mathrm{PSNR}/\log_{10}(\mathrm{params})$ measures accuracy retained per decade of parameters, requiring neither weights nor normalization.

Layer 3: profile-weighted tiers. Each axis is min–max normalized to $[0, 1]$ (higher is better) and combined by a weighted sum under a named profile (Table 2); the composite is graded $\mathrm{S} \geq 0.80$, $\mathrm{A} \geq 0.65$, $\mathrm{B} \geq 0.45$, $\mathrm{C} \geq 0.25$, else D.

Profile	Accuracy	Param. eff.	Speed	Robustness
Balanced	0.40	0.30	0.15	0.15
Quality (offline)	0.60	0.20	0.10	0.10
Compression (edge)	0.25	0.55	0.10	0.10
Realtime	0.30	0.20	0.40	0.10

Table 2: Use-case weight profiles for the composite efficiency score.

5 Experimental Setup

We use three datasets. **Synthetic mixed-structure** patches of shape $6 \times 6 \times 6$ are built from ground-truth atoms that are 40% separable (rank-one), 40% low-TT-rank, and 20% dense, combined with true sparsity 3 and Gaussian noise, so no structure is trivially favoured. **Real RGB** patches of shape $8 \times 8 \times 3$ are extracted from a standard photograph and mean-removed/normalized. **Hyperspectral (HSI)** patches of shape $6 \times 6 \times 16$ come from a synthetic cube generated by mixing a few smooth spectral signatures with smooth spatial abundance maps, giving a genuinely low-rank, fat-third-mode signal. Unless noted, dictionaries are learned on noisy training patches and evaluated by denoising PSNR against clean held-out test patches; results are averaged over random seeds. Decomposition primitives use TensorLy [10].

6 Results

6.1 Synthetic mixed-structure data

Table 3 and Figure 1 give the efficiency frontier at 12 dB. The unstructured baseline attains the highest accuracy (41.5 dB) but at the largest dictionary (25,920 parameters); CP is the most compact per atom (12× smaller) but least accurate at fixed atom count; TT and TR occupy

the middle; Tucker (here learned by a separate per-mode procedure) is astonishingly small (126 parameters, 206× compression) at the cost of accuracy.

Model	Atoms	Params	Compression	Fit (s)	PSNR (dB)
Vectorized	120	25,920	1.0×	0.85	41.49
CP	120	2,160	12.0×	3.50	33.14
Tucker	343	126	205.7×	1.42	27.01
TT	120	5,760	4.5×	1.13	38.29
TR	120	8,640	3.0×	1.24	38.95

Table 3: Synthetic mixed-structure patches ($6 \times 6 \times 6$, 12 dB, K=120, encode sparsity 6, 5 seeds).

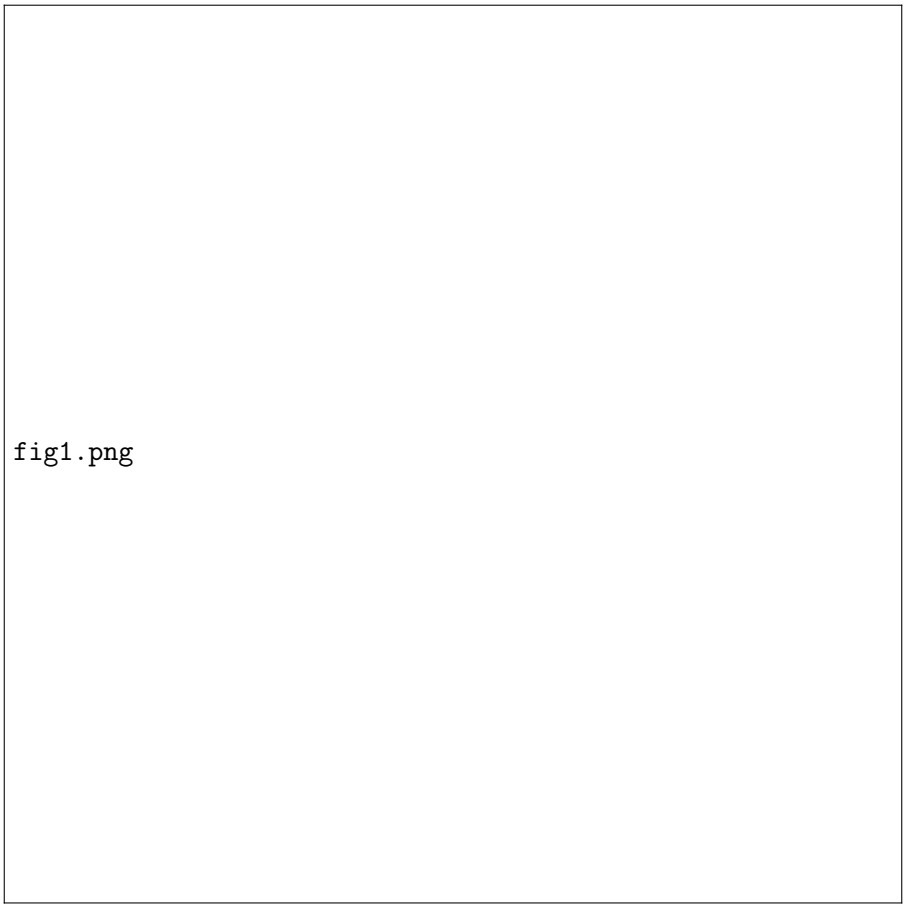


Figure 1: Efficiency frontier on synthetic data: test PSNR versus dictionary parameters (log scale). Up-and-left is better.

The rank sweep (Figure 2) is the most informative view: at fixed atom count, rank-one atoms are too weak, but as the single rank knob increases, TT reaches 42.0 dB at rank 4—exceeding the vectorized baseline (41.5 dB) at fewer parameters. CP needs rank 3 to reach 39 dB; Tucker is bounded below by separability regardless of mode width. Figure 3 reports initialization sensitivity: with data-sample initialization all methods are fairly stable, and the most constrained models show the *lowest* seed variance—consistency at a low score, not robustness worth wanting.

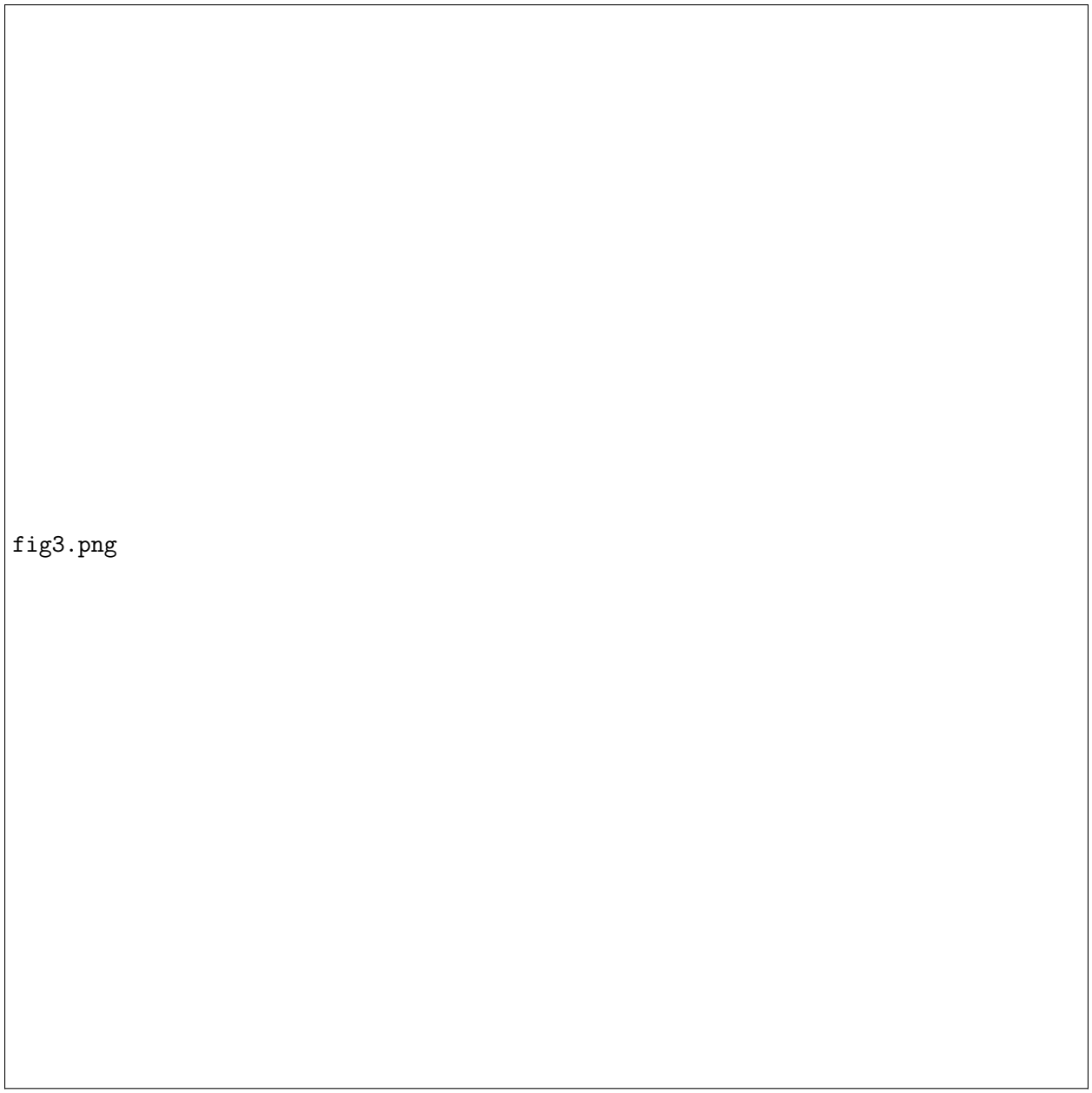


Figure 2: Rank sensitivity on synthetic data. Left: PSNR versus each method's rank knob. Right: PSNR versus dictionary parameters under the rank sweep. TT overtakes the baseline at rank 4; Tucker is capped by separability.

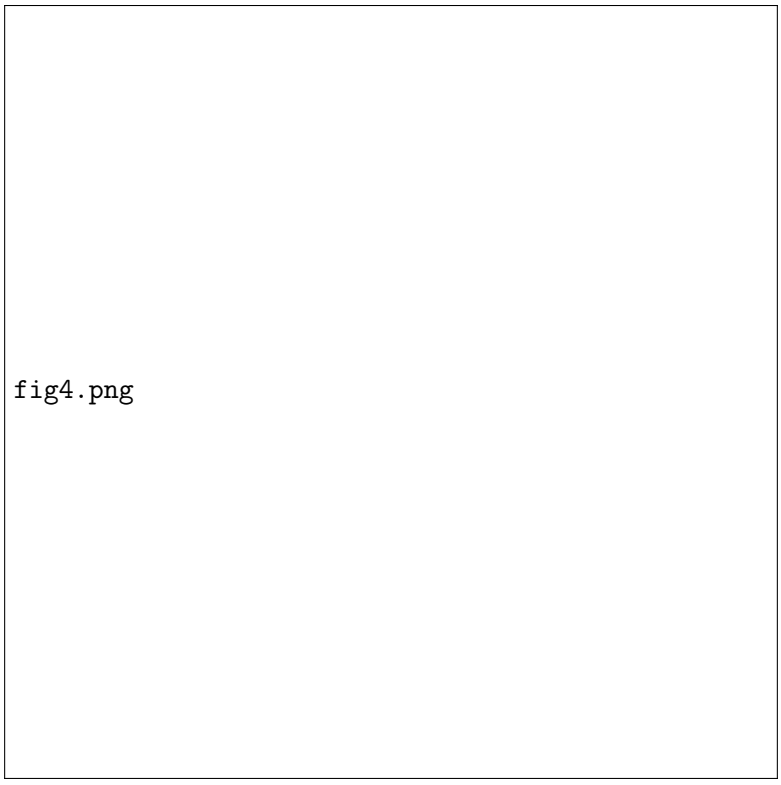


Figure 3: Initialization sensitivity across random seeds. Taller boxes are less robust; the unstructured baseline varies most, the constrained models least.

6.2 Real RGB patches: the regime flips

On real image patches (Table 4) the conclusion inverts. Every structured method except the initial Tucker *beats* the unstructured baseline while using far fewer parameters: CP reaches 31.65 dB at 10× compression versus the baseline's 30.81 dB. On real, intrinsically low-rank patches the structural constraint is no longer a handicap but a regularizer that prevents the over-parameterized free dictionary from fitting noise (Figure 4).

Model	Atoms	Params	Compression	Fit (s)	PSNR (dB)
Vectorized	100	19,200	1.0×	0.82	30.81
CP	100	1,900	10.1×	2.54	**31.65**
Tucker†	243	153	125.5×	1.41	30.74
TT	100	5,400	3.6×	1.06	31.27
TR	100	7,600	2.5×	1.18	31.27
BTD	100	4,600	4.2×	3.00	31.37

Table 4: Real RGB patches ($8 \times 8 \times 3$, 12 dB, encode sparsity 6). †Initial Tucker (separate optimizer); corrected in Section 6.3.

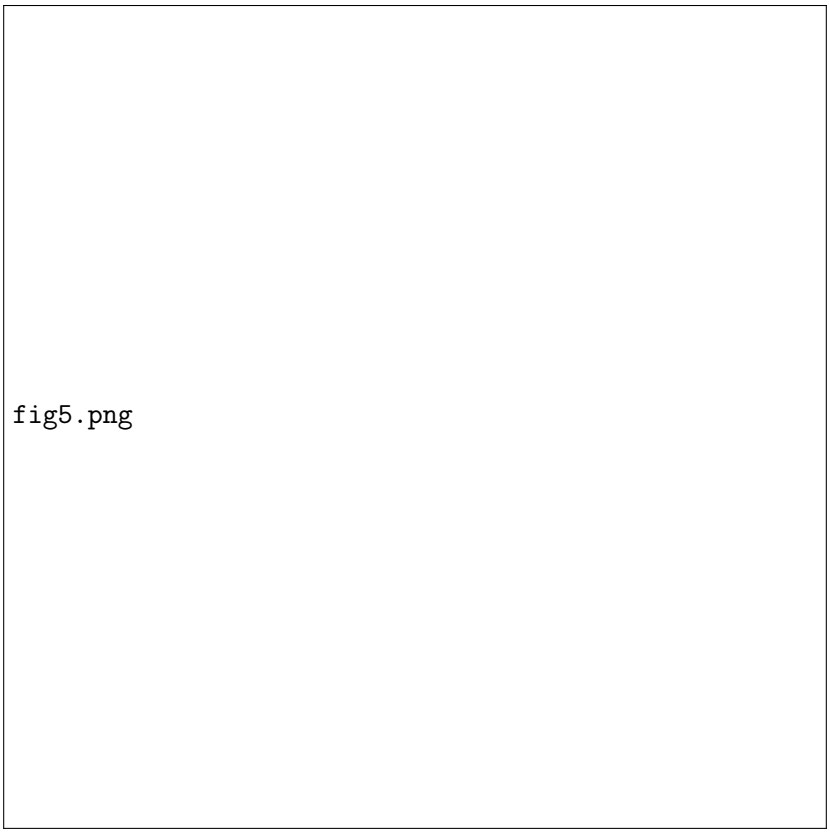


Figure 4: Efficiency frontier on real RGB patches. Structured dictionaries move above the vectorized baseline; the initial Tucker matches it at 125× fewer parameters.

CP instability. Figure 5 isolates the classic CP degeneracy. We measure a degeneracy ratio (sum of component magnitudes divided by the norm of their sum): rank-one atoms are always non-degenerate (= 1.0), and the ratio grows with rank, with random initialization producing markedly more degenerate, partially cancelling components than SVD warm-start (2.35 vs 1.62 at rank 3). Notably, reconstruction accuracy can look fine while the underlying factorization is internally degenerate—a warning if the factors themselves are to be used downstream.

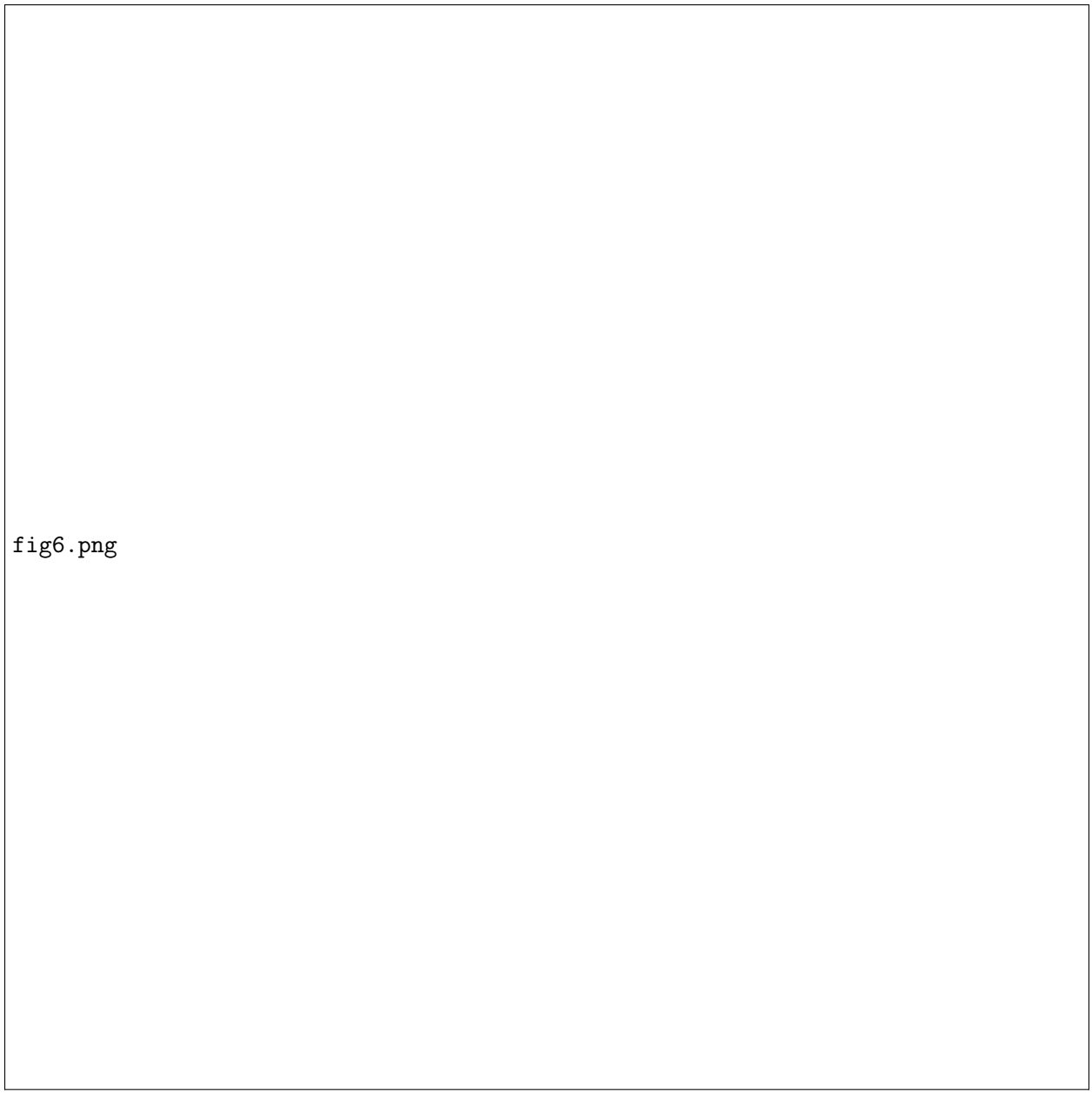


Figure 5: CP instability on real patches. Left: accuracy and seed-spread versus rank for SVD vs random initialization. Right: the degeneracy diagnostic climbs with rank and is worse under random init.

BTD interpolates CP and Tucker. Figure 6 sweeps BTD's block rank L and block count. The L=1, one-block configuration lands exactly on CP rank-one, confirming the endpoint identity, and the family traces the continuum toward Tucker. In this noisy regime the most constrained point is best; extra capacity fits noise.

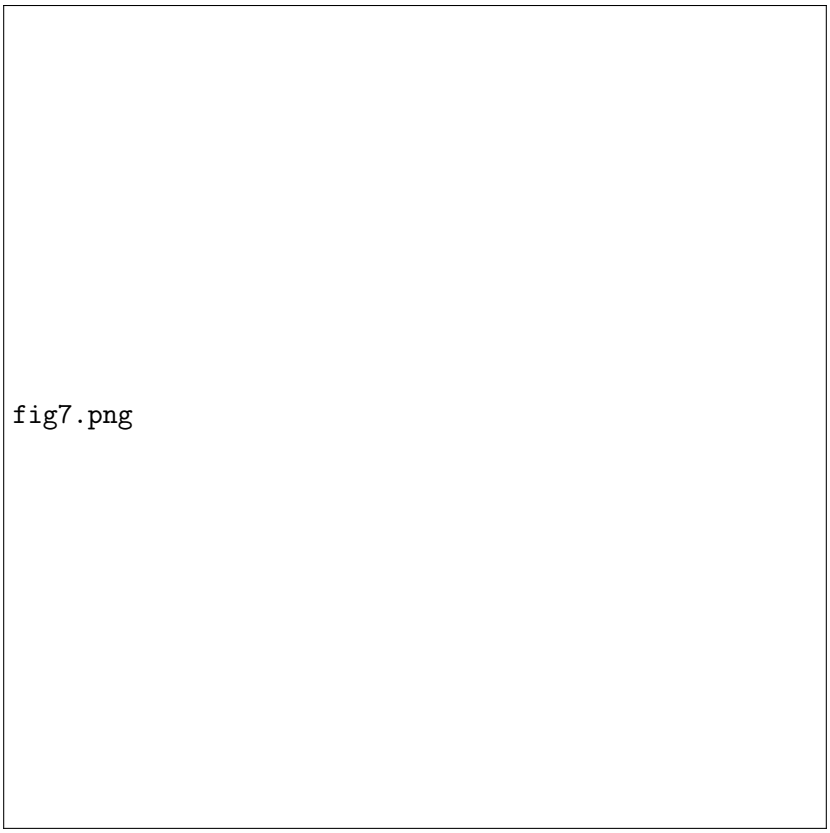


Figure 6: BTD traces the CP $\leftrightarrow$ Tucker continuum as block rank L and the number of blocks vary, overlaid on CP (rank 1–3) and Tucker reference points.

6.3 A fairness correction to Tucker

The initial Tucker of Table 4 was learned by a procedure different from the shared harness. Rebuilding Tucker into the harness (Section 3: OMP coding plus separable ALS) raises it from 30.74 to 32.16 dB at the identical 153 parameters, and *halves* its seed spread ($0.51 \rightarrow 0.09$). The earlier weakness was largely an artifact of the optimizer, not of the Tucker structure—a concrete instance of an implementation confound dominating a structural difference.

6.4 Efficiency ranking on real patches

With the corrected Tucker, the ranking system (Table 5, Figure 7) is decisive: Tucker is the sole Pareto-optimal method—highest accuracy *and* fewest parameters—so it dominates every profile (tier S), and the previously winning CP drops to tier C. The objective layer alone reduces the field: on the accuracy–parameter plane, every method other than Tucker is dominated.

Model	PSNR	Params	E	Pareto	Tier	Balanced
Tucker	32.16	153	14.72	**yes**	S	0.892
CP	31.71	1,900	9.67	no	C	0.429
TT	31.28	5,400	8.38	no	C	0.389
BTD	31.38	4,600	8.57	no	C	0.385
TR	31.31	7,600	8.07	no	C	0.306
Vectorized	30.80	19,200	7.19	no	D	0.230

Table 5: Efficiency ranking on real RGB patches (corrected Tucker, 5 seeds). $E = \text{PSNR}/\log_{10}(\text{params})$; "Balanced" is the composite score.

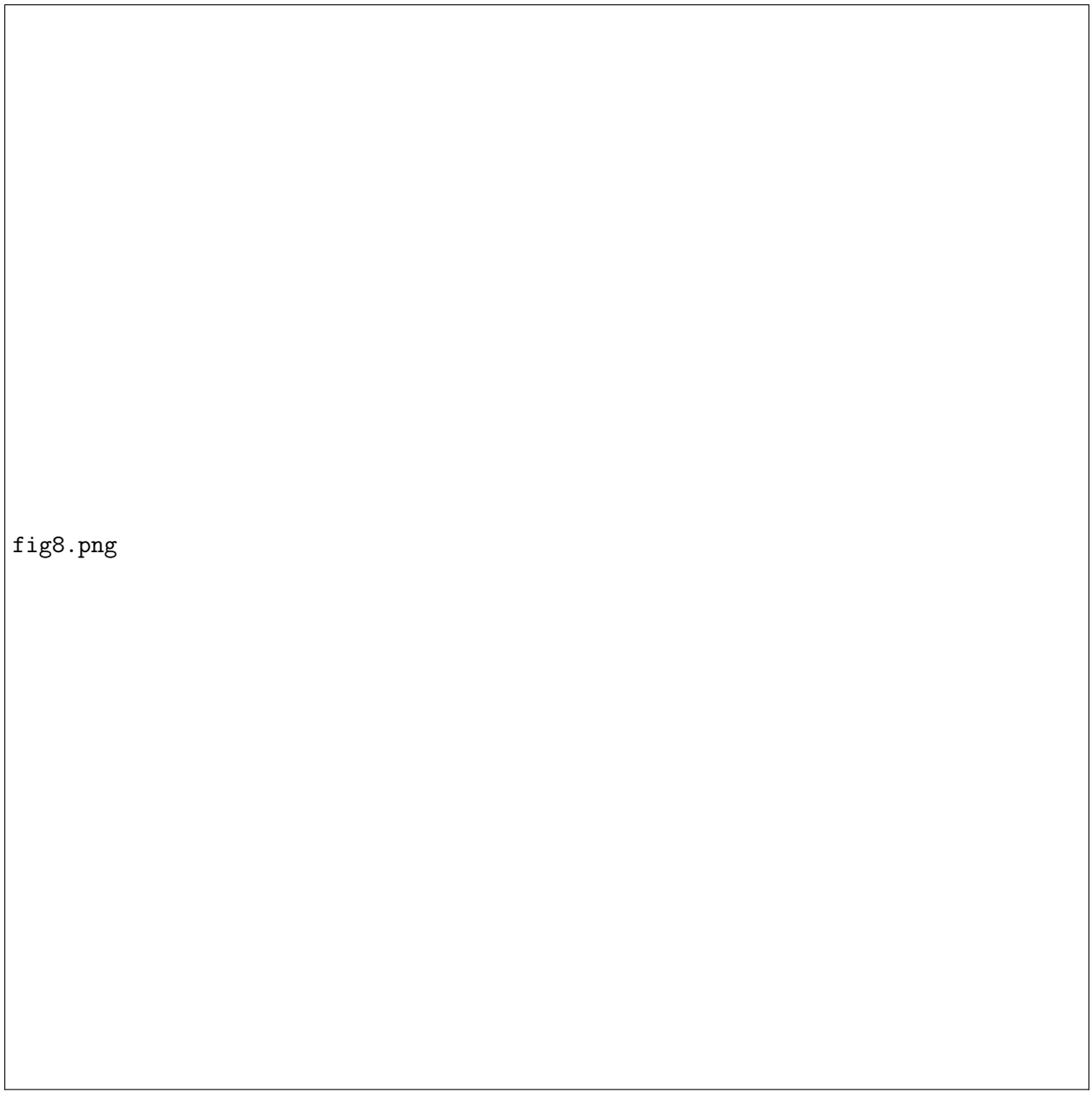


Figure 7: Ranking system. Left: objective Pareto layer (stars are non-dominated); only Tucker is on the frontier. Right: profile-weighted composite scores.

6.5 Matched-budget rate–distortion on hyperspectral data

The comparisons above use hand-chosen ranks and atom counts. To remove that confound we fix a parameter budget and let each method auto-select its rank/atom count (validation-best) to spend it, sweeping the budget to trace a rate–distortion curve, and repeating across noise levels on the HSI cube (Figure 8, Table 6). Two findings stand out. First, giving Tensor-Train a large third mode did *not* hand it the win: a single fat mode does not favour TT for DL, whose advantage is high tensor *order*, not one wide mode. Second, the ranking does not flip between heavy (6 dB) and light (18 dB) noise; only the absolute PSNR shifts. On this intrinsically low-rank cube, accuracy falls as the budget grows (clearest at 6 dB): the most regularized model wins, and the baseline

leads at 5,000 parameters partly because its atoms cost 576 each and it is forced into a few, heavily regularized atoms. Tucker sits at the far left of the curve (it cannot spend a 5,000-parameter budget without exceeding a reasonable atom cap), so "best at 5,000 parameters" understates its per-parameter efficiency.

Model (at ≈5,000 params)	6 dB PSNR	18 dB PSNR
Vectorized	**32.27**	**41.80**
CP	29.91	41.08
TT	29.24	40.58
TR	29.16	40.54
BTD	28.49	40.21
Tucker*	26.90	38.82

Table 6: Matched-budget ranking on the HSI cube ($6 \times 6 \times 16$) at the ≈5,000-parameter slice. *Tucker operates at 304 parameters (it cannot reach the budget); see text.

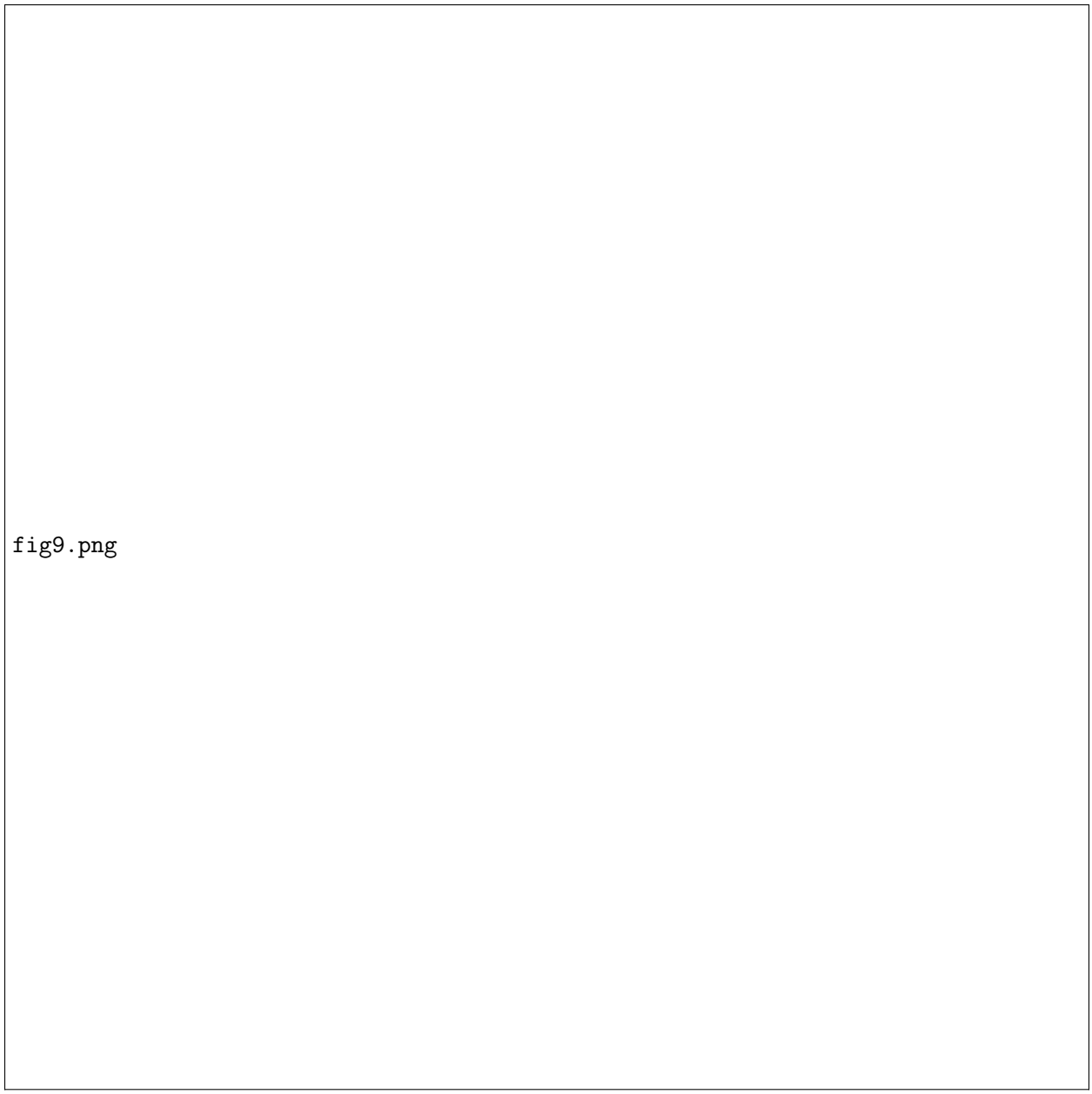


Figure 8: Matched-budget rate–distortion on the HSI cube at 6 dB (heavy) and 18 dB (light) noise. Each point auto-selects its rank/atoms to hit the budget; the dotted line marks the 5,000-parameter slice. The ordering is stable across noise, and a fat third mode does not favour TT/TR.

7 Discussion

Three themes emerge. **(i) Regime dependence.** There is no absolute best decomposition: the unstructured baseline wins on rich clean synthetic data; structured dictionaries win on noisy real patches where their constraint regularizes; and at matched budget on low-rank HSI, few full atoms or a tiny Tucker are most efficient. Which decomposition to use is a function of signal order, noise, and intrinsic rank, not a fixed ranking. **(ii) Confounds can dominate structure.** The Tucker rebuild (Section 6.3) moved it from last to first at identical size purely by fixing the optimizer,

and the matched-budget study revised the earlier "structured wins" story by revealing that some of that advantage was really the freedom to buy many cheap atoms. **(iii) Matched-parameter is not matched-atom.** At equal storage, the baseline is forced into few heavily regularized atoms while structured methods spend the same budget on many atoms; which budget is "fair" depends on whether one cares about storage or per-atom quality.

8 Limitations

We are explicit about scope. The CP/TT/TR/BTD methods are *projection variants* inside a shared K-SVD loop, not the canonical published algorithms (e.g. jointly optimized factors, PALM-based TT-DL, or SeDiL); the numbers rank these implementations, and the labels should not be read as those methods' definitive performance. There is no single "correct" algorithm per decomposition—each admits many optimizers—so a fully faithful leaderboard would report several implementations per structure and show the within-structure spread. Our datasets are modest (hundreds of patches, single images, few noise levels), the error bars capture only seed variance, and the HSI cube is synthetic. The matched-budget study caps atom counts for tractability, which bounds Tucker artificially. Finally, the harness materializes the full effective dictionary and runs identical OMP for fairness, which discards the fast structured transforms that are a genuine computational advantage of Kronecker/TT dictionaries; the reported "speed" axis therefore understates the tensor methods. These are the natural next steps rather than flaws to hide.

9 Conclusion

We presented a single harness in which five tensor-structured dictionaries and an unstructured baseline differ only by how each atom is parameterized, connected them to dictionary learning through the Khatri–Rao (CP) and Kronecker (Tucker) forms, and built a three-layer efficiency ranking system. Across synthetic, real, and hyperspectral data the best decomposition is regime-dependent; implementation confounds can outweigh structural differences; and a matched parameter budget both tightens and, in places, reverses naive conclusions. The framework is designed so that new decompositions are one projector and one parameter rule away, and so that the ranking can be re-derived on any dataset.

References

[1] M. Aharon, M. Elad, and A. Bruckstein. K-SVD: An algorithm for designing overcomplete dictionaries for sparse representation. *IEEE Transactions on Signal Processing*, 2006.

[2] Y. C. Pati, R. Rezaiifar, and P. S. Krishnaprasad. Orthogonal matching pursuit: recursive function approximation with applications to wavelet decomposition. *Asilomar Conference on Signals, Systems and Computers*, 1993.

[3] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 2009.

[4] L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 1966.

[5] I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 2011.

[6] Q. Zhao, G. Zhou, S. Xie, L. Zhang, and A. Cichocki. Tensor ring decomposition. *arXiv:1606.05535*, 2016.

[7] L. De Lathauwer. Decompositions of a higher-order tensor in block terms—Part II: Definitions and uniqueness. *SIAM Journal on Matrix Analysis and Applications*, 2008.

[8] S. Hawe, M. Seibert, and M. Kleinsteuber. Separable dictionary learning. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013.

[9] C. F. Caiafa and A. Cichocki. Computing sparse representations of multidimensional signals using Kronecker bases. *Neural Computation*, 2013.

[10] J. Kossaifi, Y. Panagakis, A. Anandkumar, and M. Pantic. TensorLy: Tensor learning in Python. *Journal of Machine Learning Research*, 2019.